Studio Nx

JS 06 — isLoggedIn()

2026-10-07

JS 06 — `isLoggedIn()`

항목	내용
파일	`studionx/frontend/static/js/core/auth.js`
작업	P1-02 프론트 공통 JS · PRD 01 FR-105
작성	Claude (AI) · 2026-10-06 · 사용자 검토
이전 · 다음	05 `parseLoginHash()` · 07 `requireLogin()`

1. 하는 일

지금 로그인 상태인지 `true` · `false` 로 알려 준다. 토큰이 없으면 리프레시 쿠키로 다시 받아 본 뒤 판단한다.

2. 언제 쓰나

상황	부르는 곳
로그인이 필요한 페이지(마이페이지 · 내 프로젝트 · 회원 정보 · 시퀀서)에 들어올 때	07 `requireLogin()` — `false` 면 로그인 화면으로 보낸다
페이지를 열 때, 맨 위 메뉴 줄(헤더) 오른쪽에 어떤 버튼을 보여 줄지 정할 때	`auth-ui.js` `initAuthState()` (P1-07) — 로그인 상태면 **My Page · My Projects** 버튼을, 아니면 **Sign In · Join Us** 버튼을 보여 준다
`home.html` 을 열 때	이미 로그인 상태면 `#login` 모달을 열지 않는다 (P1-07)

헤더 버튼이 바뀌는 예

헤더는 홈 · 가입 · 마이페이지 등 모든 페이지 맨 위에 똑같이 들어가는 메뉴 줄이다 (`static/partials/header.html`). 오른쪽 버튼 네 개가 모두 HTML 에 들어 있고, 이 함수의 결과에 따라 두 개만 보이게 한다.

로그인 상태	보이는 버튼	숨기는 버튼
로그인 안 됨 (`false`)	Sign In · Join Us	My Page · My Projects
로그인 됨 (`true`)	My Page · My Projects	Sign In · Join Us

버튼에는 `data-auth-show="out"` (로그인 안 됐을 때 보임) · `data-auth-show="in"` (로그인했을 때 보임) 표시가 붙어 있다.

로그인 상태를 판단하는 기준

서버에 "로그인 중"이라는 기록은 없다. 브라우저가 가진 토큰으로 판단한다.

브라우저 상태	판단
메모리에 Access Token 이 있다	로그인 상태
Access Token 은 없지만 리프레시 쿠키가 유효하다 (페이지를 옮긴 직후 · 자동 로그인)	재발급에 성공하면 로그인 상태
둘 다 없거나 리프레시 쿠키가 만료되었다	로그인 안 된 상태

3. 코드

import (파일 맨 위)

```
import { api, clearAccessToken, getAccessToken, refreshAccessToken } from './api.js';
```

`auth.js` 맨 위의 import 줄이다. 이 함수는 이 중 **`getAccessToken`** · **`refreshAccessToken`** 을 쓴다 (01).

함수

```
export async function isLoggedIn() {
  if (getAccessToken()) return true;
  return refreshAccessToken();
}
```

문법 풀이

import 문법	뜻
`import { a, b } from './api.js'`	`api.js` 가 `export` 한 것 중 `a` · `b` 를 이름 그대로 가져옴. `'./'` 는 같은 폴더. 브라우저는 `<script type="module">` 일 때만 `import` 를 읽는다

식	뜻
`async function`	결과를 Promise 로 돌려주는 함수. 부르는 쪽은 `await isLoggedIn()` 으로 `true` · `false` 를 받는다
`if (getAccessToken())`	토큰 문자열이 있으면 참, `null` 이면 거짓
`return refreshAccessToken();`	재발급 결과(`true` · `false`)를 그대로 돌려준다. `async` 함수 안에서 Promise 를 돌려주면 그 결과가 이 함수의 결과가 된다

4. 흐름

1. 메모리에 Access Token 이 있으면 서버에 묻지 않고 `true`.
2. 없으면 `refreshAccessToken()` 을 부른다 (01).
3. 재발급에 성공하면 `true`, 실패하면 `false`.

5. 왜 이렇게

선택	이유
토큰이 있으면 바로 `true`	같은 페이지 안에서 여러 번 불려도 서버 요청이 생기지 않는다
토큰의 만료 시간은 확인하지 않음	만료된 토큰으로 API 를 부르면 `apiFetch()` 가 401 을 받고 재발급한 뒤 다시 보낸다. 같은 처리를 두 곳에 두지 않는다
재발급은 `refreshAccessToken()` 에 맡김	헤더 버튼을 정하는 코드와 `requireLogin()` 이 페이지를 열 때 동시에 불러도 재발급 요청은 한 번만 나간다 (01 의 진행 중인 Promise 공유)
실패해도 예외를 던지지 않음	네트워크 오류도 "로그인 안 됨"으로 본다. 부르는 쪽은 `true` · `false` 만 확인한다

6. 시험 (fetch 를 가짜로)

경우	결과
메모리에 토큰이 있음	`true`, 서버 요청 0번
토큰 없음 · 리프레시 쿠키 유효	`true`, 재발급 1번, 새 토큰 저장
토큰 없음 · 리프레시 쿠키 없음 (401)	`false`, 토큰 없음
네트워크 오류	`false`, 예외 없음
동시에 두 번 호출 (헤더 버튼 정하기 + `requireLogin`)	둘 다 `true`, 재발급 1번